



PHINS — Global Health Marketplace Architecture

PHINS Global Health Marketplace Architecture

Purpose

This document defines the target architecture for PHINS as a health-insurance-backed marketplace that connects:

- customers with active PHINS health wallets
- approved internal and external suppliers
- PHINS claims and policy systems
- external payment processors
- external insurers and claims clearing networks
- BI and AI assessment pipelines

For the execution-ready build plan derived from this target state, see `docs/health_marketplace_implementation_spec.md`.

It is grounded in the current repository shape:

- customer experience in `web_portal/static/dashboard.html`
- supplier experience in `web_portal/static/supplier-portal.html`
- admin oversight in `web_portal/static/admin.html` and `web_portal/static/admin-supplier-dashboard.html`
- routing and orchestration in `web_portal/server.py`
- analytics in `services/bi_analytics_service.py`
- integrity validation in `services/platform_integrity_service.py`
- supplier marketplace persistence in `database/models.py`
- audit and append-only lineage in `docs/platform_data_architecture.md`

Executive conclusion

PHINS should evolve into a single, ledger-governed marketplace operating core with one canonical transaction chain from:

- 1 eligibility and quote
- 2 wallet authorization or insurer authorization
- 3 order fulfillment
- 4 supplier settlement

- 5 markup recognition
- 6 insurer reimbursement or refund
- 7 BI/AI monitoring

The most important design decision is to remove the current split between in-memory marketplace and durable insurance records. The target state is:

- one durable marketplace domain in Postgres
- one double-entry wallet and settlement ledger
- one event/outbox pipeline for BI, AI, actuarial, and reconciliation use
- one customer journey embedded in the current dashboard
- one supplier operating journey embedded in the current supplier portal
- one adjudication and finance control plane across PHINS, suppliers, and external payers

Current architecture snapshot

What already exists

The repository already contains valuable building blocks:

- customer wallet and marketplace entry points in `dashboard.html`
- supplier onboarding, offers, and orders in the supplier services and dashboard files
- `Supplier`, `SupplierOffer`, and `SupplierOrder` ORM models in `database/models.py`
- admin supplier analytics and approval flows
- claim payment to wallet logic in `web_portal/server.py`
- platform event ledger and integrity validation services
- BI services that already compute supplier, claims, and financial indicators

Structural gaps that prevent a world-class marketplace

1 Split source of truth

- health wallets, supplier registry snapshots, and some marketplace activity still run from in-memory dictionaries
- claims and platform ledger have stronger persistence semantics than wallets and settlements

1 Insufficient financial separation

- customer wallet spend, PHINS markup, supplier payable, insurer receivable, refunds, and reserves are not yet represented as separate sub-ledgers with strict posting rules

1 Supplier settlement is incomplete

- supplier payout exists conceptually, but not as a durable settlement engine with holds, release rules, chargebacks, dispute reserve, and reconciliation

1 External suppliers and insurers are not first-class integrations

- there is no canonical connector framework for catalog sync, availability sync, order routing, claim reimbursement, remittance, and refund orchestration

1 Markup accounting is not fully institutionalized

- pricing logic can compute margin, but margin recognition must feed the balance sheet and income statement with explicit journal treatment

1 AI/BI lacks a canonical marketplace event model

- analytics exist, but the system needs one event vocabulary for conversion, fraud, profitability, medical utilization, supplier performance, and insurer reimbursement outcomes

Target operating model

Customer promise

Every customer should be able to:

- open `dashboard.html`
- see real-time wallet, coverage, deductible, insurer coverage, and PHINS credit line
- search consultations, devices, daily supplies, pharmacy, home care, diagnostics, labs, and more
- receive ranked offers across PHINS-native and external suppliers
- pay using wallet, card, employer credit, or external insurer pre-authorization
- receive delivery, service scheduling, or telehealth confirmation
- receive automatic reimbursement or refund when an external insurer covers the event
- see a tamper-evident activity trail for every approval, charge, payout, claim, and refund

Supplier promise

Every supplier should be able to:

- onboard once with KYC/KYB, licensing, and payout details
- publish a structured catalog and availability
- receive orders with coverage and payment certainty indicators
- get settled according to contract terms
- view disputes, deductions, and remittances transparently
- integrate by API, flat file, webhook, or operator portal

PHINS finance promise

PHINS finance and risk teams should always know:

- gross merchandise value
- recognized revenue
- deferred revenue
- supplier payable
- insurer receivable
- refund liability
- claims reserve movement
- loss ratio and contribution margin by customer, supplier, category, region, and payer

Architecture principles

1 Ledger first

- no wallet movement, markup, claim payment, supplier payout, refund, or insurer remittance exists without a corresponding immutable financial event

1 Marketplace and insurance are one operating system

- the marketplace must consume policy, coverage, deductible, network, and claim eligibility data directly rather than bolting them on later

1 Authorizations before captures

- use wallet holds, card authorizations, insurer pre-authorizations, or purchase orders before fulfillment

1 Idempotency everywhere

- every external callback and every payment, order, settlement, and refund mutation must carry an idempotency key

1 Outbox for all strategic events

- OLTP writes and event publication must be transactionally linked

1 Reconciliation is a product feature

- PHINS should surface ledger health, unmatched receivables, supplier variances, and insurer remittance exceptions in dashboards

1 Coverage-aware ranking

- the cheapest offer is not always the best offer; ranking must consider coverage, SLA, medical suitability, fraud risk, patient preference, and total landed cost

Target bounded contexts

1. Identity, trust, and network governance

Responsibilities:

- customer identity
- supplier KYB, licensing, sanctions screening
- payer network participation
- consent and authorization
- tenant isolation

Core entities:

- Customer
- Supplier
- SupplierCredential
- License

- NetworkContract
- ExternalPayer
- ConsentGrant

2. Coverage and eligibility

Responsibilities:

- PHINS policy validation
- external insurance discovery
- benefits lookup
- deductible and co-pay rules
- pre-authorization workflows

Core entities:

- Policy
- CoveragePlan
- EligibilityCheck
- PreAuthorization
- BenefitRule
- NetworkRule

3. Marketplace search, quote, and orchestration

Responsibilities:

- supplier catalog normalization
- inventory and schedule availability
- offer ranking
- cart, quote, and order construction
- order orchestration

Core entities:

- CatalogItem
- Offer
- Quote
- Cart
- Order
- OrderItem
- ServiceAppointment
- DeliveryShipment

4. Wallet, payments, and accounting

Responsibilities:

- health wallet balances
- holds and captures
- double-entry postings
- external PSP payments
- supplier payables
- refund liabilities
- markup revenue recognition

Core entities:

- WalletAccount
- WalletHold
- LedgerEntry
- PaymentIntent
- Charge
- Refund
- SupplierSettlement
- GeneralJournalEntry

5. Claims and external payer clearing

Responsibilities:

- claim as service
- direct claim filing to external insurers
- remittance and EOB ingestion
- reimbursement to customer or PHINS
- coordination of benefits

Core entities:

- MarketplaceClaim
- PayerCase
- CoverageDecision
- RemittanceAdvice
- PayerReceivable
- RecoveryCase

6. Intelligence and optimization

Responsibilities:

- BI dashboards
- AI pricing and ranking
- fraud and waste detection
- demand forecasting

- supply quality scoring
- reimbursement leakage analytics

Core entities:

- MarketplaceEvent
- FeatureSnapshot
- ModelVersion
- Recommendation
- FraudAlert
- ProfitabilityView

Canonical data model

The target domain model should extend the existing supplier schema and add the missing durable financial and payer objects.

Core customer and supplier objects

- Customer
- Supplier
- SupplierOffer
- SupplierOrder
- SupplierCatalogVersion
- SupplierSchedule
- SupplierNetworkContract
- ExternalPayer
- ExternalPayerPlan

Wallet and accounting objects

- wallet_accounts
- one row per customer and wallet type
- available balance, held balance, currency
- wallet_holds
- hold amount, status, expiry, idempotency key
- ledger_entries
- append-only, double-entry postings
- payment_intents
- wallet, card, bank, insurer, mixed-tender
- supplier_settlements
- gross sales, markup, supplier payout, reserve holdback, status
- journal_entries
- accounting view of recognized revenue, payable, receivable, refund liability

Insurance and reimbursement objects

- coverage_checks
- pre_authorizations
- marketplace_claims
- payer_submissions
- remittance_advices
- payer_receivables
- coordination_of_benefits_cases

Governance and traceability objects

- audit_logs
- platform_ledger_entries
- outbox_events
- reconciliation_runs
- pricing_explanations
- model_decisions

Ledger and accounting architecture

Wallet and marketplace posting model

Every marketplace purchase should create explicit postings:

At order authorization

- debit: customer available wallet balance
- credit: customer held wallet balance

If external card is used:

- debit: PSP clearing receivable
- credit: customer payment pending

If external insurer pre-authorizes:

- debit: payer receivable pending authorization
- credit: authorized coverage reserve

At supplier acceptance and capture

- debit: held wallet balance
- credit: marketplace clearing

Then split the clearing amount into:

- credit: supplier payable
- credit: PHINS markup revenue or deferred revenue

- credit: tax payable if relevant
- credit: delivery payable when applicable

This makes markup visible and auditable.

Markup and margin treatment

To make markups valid and visible on the balance sheet and management views:

1 Store both:

- gross_sales_amount
- supplier_cost_amount
- markup_amount
- markup_percent

1 Enforce:

- $\text{markup_amount} = \text{gross_sales_amount} - \text{supplier_cost_amount}$
- $\text{markup_percent} = \text{markup_amount} / \text{supplier_cost_amount} * 100$

1 Post the markup into dedicated accounts:

- marketplace revenue
- deferred marketplace revenue when performance obligation is not complete
- marketplace contra-revenue for refunds and rebates

1 Reflect it in dashboards:

- gross merchandise value
- net revenue
- contribution margin
- realized profit margin
- refund-adjusted margin

Supplier settlement model

Supplier payouts must never be a side effect of order completion alone. They must be processed through a settlement engine with:

- order-level settlement rows
- reserve holdback for disputes and returns
- configurable settlement frequency by supplier
- remittance document generation
- payout rail tracking
- settlement reconciliation against bank or PSP files

Settlement formula:

`net_supplier_payout = gross_sales - markup - delivery_fee_share - penalties - reserve_holdback + supplier_adjustments`

Refund and reversal model

Refunds must support:

- customer cancellation before fulfillment
- supplier cancellation
- failed delivery
- insurer retro denial
- duplicate charge
- fraud reversal

Every refund must create:

- order reversal event
- wallet or PSP reversal posting
- markup reversal
- supplier payable reduction or settlement clawback
- refund liability reduction once executed

External supplier integration architecture

Supplier connectivity modes

PHINS should support four integration tiers:

1 Portal-managed suppliers

- manual catalog entry in supplier portal
- best for small clinics and pharmacies

1 API-native suppliers

- REST or GraphQL for catalog, pricing, inventory, appointments, and order updates

1 EDI or flat-file suppliers

- scheduled CSV/SFTP ingestion for price lists and remittances

1 Aggregator connectors

- one connector to reach multiple pharmacies, labs, telehealth, or logistics networks

Canonical connector interfaces

- CatalogConnector
- AvailabilityConnector
- OrderConnector
- FulfillmentConnector

- InvoiceConnector
- SettlementConnector
- RefundConnector

Each connector must publish canonical events into PHINS, never raw external payloads directly into downstream services.

Normalization layer

All external supplier data must be normalized into:

- unified category taxonomy
- product/service master
- unit and pack normalization
- regional SLA model
- medical compliance flags
- substitution and equivalency groups

This is critical for accurate search, ranking, and profitability.

External insurer and claim-as-a-service architecture

PHINS should support two reimbursement patterns:

A. Customer-pay-first, reimburse-later

- 1 customer pays with wallet or mixed tender
- 2 PHINS files claim to external insurer
- 3 insurer remits to PHINS or customer
- 4 PHINS auto-credits customer wallet or closes payer receivable

B. PHINS-advance, insurer-recover

- 1 PHINS pays supplier immediately
- 2 PHINS records payer receivable
- 3 external insurer adjudicates later
- 4 remittance clears receivable
- 5 denial triggers customer co-pay collection or write-off workflow

Claim-as-a-service capabilities

- benefits check before checkout
- pre-authorization submission before order commit
- claim packet generation from order, diagnosis, provider, invoice, and proof of delivery
- remittance ingestion
- exception queue for denials and underpayments
- appeal workflow
- recovery analytics

Refund and insurer interaction rules

If insurer covers an already paid service:

- refund should be directed based on payment source priority:
 - 1 customer wallet reimbursement
 - 2 customer card reimbursement
 - 3 PHINS receivable offset
 - 4 employer or sponsor reimbursement

If insurer denies after PHINS has advanced funds:

- hold customer liability separately from fraud or supplier fault
- never silently rewrite settled ledger entries
- create compensating entries only

BI and AI architecture

Event model

Every strategic action must emit a versioned event:

- `eligibility.checked`
- `quote.generated`
- `quote.accepted`
- `wallet.hold_created`
- `order.created`
- `order.confirmed`
- `order.fulfilled`
- `settlement.calculated`
- `settlement.paid`
- `claim.submitted`
- `claim.adjudicated`
- `remittance.received`
- `refund.created`
- `refund.completed`
- `integrity.violation_detected`

BI dashboards

Extend the current dashboard surfaces with role-specific metrics:

Customer dashboard (dashboard.html)

- wallet available, held, and reimbursable balances
- insurance coverage applied to each order

- reimbursement timeline and pending refunds
- order and claim status timeline

Supplier portal (supplier-portal.html)

- fill rate
- order acceptance rate
- on-time fulfillment
- dispute rate
- payout aging
- net settlement by period

Admin supplier dashboard

- category-level GMV
- supplier margin contribution
- outstanding settlements
- payout failures
- external connector health
- insurer receivables aging

Finance and actuarial dashboards

- claims-service recovery rate
- reimbursement turnaround
- gross-to-net margin bridge
- loss ratio by category and payer
- denied reimbursement leakage
- fraud loss avoided

AI services

AI should improve, not override, core controls.

Ranking AI

Uses:

- price
- coverage fit
- geography
- SLA
- historical success
- supplier quality
- fraud risk
- clinical appropriateness

Finance and risk AI

Uses:

- abnormal margin detection
- duplicate reimbursement detection
- supplier collusion patterns
- phantom fulfillment detection
- anomalous return/refund rates
- payer underpayment prediction

Demand and network AI

Uses:

- demand forecasting by category and geography
- supplier onboarding prioritization
- inventory pressure alerts
- benefit design optimization

All model outputs must store:

- model version
- feature timestamp
- explanation payload
- confidence score
- override reason if human action differs

Integrity and control architecture

Non-negotiable controls

1 Double-entry wallet ledger

- balances are derived, not manually edited

1 Append-only platform ledger

- all strategic and financial events chain with hashes

1 Idempotency store

- required for supplier callbacks, insurer remittances, and payment retries

1 Transactional outbox

- event publication cannot diverge from OLTP commit

1 Reconciliation jobs

- wallet-to-ledger
- order-to-settlement

- claim-to-remittance
- PSP-to-bank
- supplier payable-to-payout

1 Data contracts

- canonical schemas for offers, orders, claims, reimbursements, settlements, and refunds

1 Cross-tenant and cross-customer isolation

- no supplier sees another supplier's orders
- no customer sees any unrelated wallet or claim
- payer integrations scoped by contract and consent

Referential integrity rules

- every order references a valid customer and supplier
- every payment references a single payment intent
- every hold capture references an existing hold
- every settlement references one or more fulfilled order items
- every reimbursement references a claim or coverage event
- every refund references original order and original funding source

Balance reconciliation rules

- wallet posted balance = credits - debits - active holds
- supplier payable balance = unsettled captured order payouts - paid settlements - clawbacks
- payer receivable = submitted covered amount - remitted amount - write-offs
- balance sheet marketplace revenue = recognized markups - contra revenue

Recommended target APIs

Marketplace APIs

- GET /api/marketplace/search
- POST /api/marketplace/quotes
- POST /api/marketplace/orders
- POST /api/marketplace/orders/{id}/cancel
- GET /api/marketplace/orders/{id}

Wallet and payment APIs

- POST /api/wallet/payment-intents
- POST /api/wallet/holds
- POST /api/wallet/holds/{id}/capture
- POST /api/wallet/holds/{id}/release
- POST /api/wallet/refunds

Supplier APIs

- POST /api/suppliers/connectors/catalog-sync
- POST /api/suppliers/connectors/order-callback
- POST /api/suppliers/settlements/run
- GET /api/suppliers/settlements/{id}

External payer APIs

- POST /api/payers/eligibility/check
- POST /api/payers/preauth
- POST /api/payers/claims
- POST /api/payers/remittances/import
- POST /api/payers/refunds/reconcile

Integrity and analytics APIs

- GET /api/integrity/marketplace
- GET /api/bi/marketplace-executive
- GET /api/bi/marketplace-finance
- GET /api/bi/marketplace-supplier
- GET /api/ai/marketplace-insights

Recommended repository evolution in this codebase

Near-term changes aligned to existing structure

- 1 Add repository-backed persistence for:
 - health wallets
 - wallet holds
 - wallet ledger entries
 - supplier settlements
 - payer receivables
 - remittances
- 1 Keep the current portals but move write paths behind service and repository boundaries:
 - web_portal/server.py remains the HTTP surface
 - service layer owns workflow logic
 - repositories own durable state transitions
- 1 Introduce a canonical marketplace accounting service:
 - computes markup
 - posts journal entries

- updates balance sheet views

1 Extend the current integrity services to validate:

- wallet holds versus captures
- settlement aging
- markup recognition consistency
- payer receivable aging
- refund lineage

1 Emit a shared marketplace event schema for BI and AI consumers.

Dashboard placement strategy

Keep the existing branch and dashboard topology, but deepen each role:

- `dashboard.html`
- customer shopping, benefits preview, reimbursement tracking
- `supplier-portal.html`
- catalog, order operations, payout and remittance views
- `admin-supplier-dashboard.html`
- supplier governance, connector health, payout operations
- `admin.html`
- enterprise BI, marketplace controls, insurer receivables
- actuarial and accounting dashboards
- profitability, reserve adequacy, recovery, fraud, reimbursement leakage

Implementation roadmap

Phase 1: unify source of truth

- persist health wallet and wallet ledger tables
- stop creating net-new in-memory financial records as system of record
- keep in-memory compatibility only as cache or test adapter

Phase 2: payment integrity

- introduce payment intents, holds, captures, and refunds
- make every marketplace order wallet-safe and idempotent
- separate gross sales, PHINS markup, and supplier payout

Phase 3: supplier settlement

- create settlement runs, remittances, and payout files
- add settlement dashboarding and reconciliation

Phase 4: external supplier APIs

- build connector framework
- normalize external catalogs and availability
- implement webhooks with signature verification and replay protection

Phase 5: external insurer recovery

- build eligibility, pre-auth, claim submission, and remittance ingestion flows
- add payer receivable accounting and denial workflows

Phase 6: BI and AI operating system

- publish canonical events to the lakehouse
- add finance, marketplace, supplier, and reimbursement domain metrics
- deploy AI risk, pricing, and reimbursement anomaly models

Final architecture decision

The optimal PHINS architecture is not a separate marketplace beside insurance.

It is a unified health-financial operating system where:

- the customer wallet is the checkout and reimbursement center
- suppliers are controlled network participants
- markup is accounted for as explicit revenue with contra-revenue and payout controls
- external insurers are integrated as recovery and coverage engines
- BI and AI run off the same immutable event and ledger backbone
- every balance can be reproduced from append-only postings

That is the architecture most likely to let PHINS build a globally scalable, insurance-backed health marketplace with flawless data integrity.